

JSON Output Requirements

1. Purpose

The system must use a consistent JSON structure so that information collected from blockchain APIs, scam detectors and the LLM can be passed between different parts of the application in a set format.

The JSON output should clearly identify:

- The input being analysed.
 - The type of blockchain input/address.
 - Which APIs or data sources were checked.
 - The data returned from each check.
 - Any failed or skipped checks.
 - The final scam risk evaluation, when one has been performed.
 - The evidence supporting the risk evaluation.
-

2. Top Level JSON Structure

The combined JSON output should contain the following main sections:

- input
- address_analysis
- api_checks
- risk_evaluation
- summary

Example structure:

```
{
  "input": {},
  "address_analysis": {},
  "api_checks": {},
  "risk_evaluation": {},
  "summary": {}
}
```

The same overall structure should be maintained wherever possible so that the other components do not need to interpret different JSON formats for different requests.

3. Input Information

The JSON must identify the original value submitted for analysis.

Required fields:

```
"input": {
  "value": "0x...",
  "input_type": "address",
  "chain": "ethereum"
}
```

Display / Data Rules

- value contains the original blockchain input.
- input_type identifies what was actually submitted.

Possible values may include:

- address
- transaction_hash
- token_contract
- token_name
- chain identifies the blockchain where known.
- If the blockchain cannot yet be identified, an appropriate null value should be used rather than just guessing.

4. Address Type

The JSON must contain the result of address/input classification.

Example:

```
"address_analysis": {
  "chain_family": "evm",
  "address_type": "wallet"
}
```

Possible address types may include:

- wallet
- delegated_wallet
- contract
- token_contract
- unknown

The result should distinguish between the blockchain family and the identified address type.

If the system cannot confidently determine the address type, it should return unknown rather than assigning an unsupported classification.

5. APIs / Checks Performed

The JSON must make it possible to determine which external checks or data sources were used.

Each check should contain:

- Name of the check/fetcher.
- Status.
- Returned data where successful.
- Error/reason where unsuccessful.
- Source/provider where appropriate.

Example:

```
"api_checks": {  
  "contract": {  
    "status": "pass",  
    "source": "Etherscan",  
    "data": {},  
    "error": null  
  },  
  
  "transactions": {  
    "status": "pass",  
    "source": "Etherscan",  
    "data": [],  
    "error": null  
  },  
  
  "liquidity": {  
    "status": "skip",  
    "source": "DexScreener",  
    "data": null,  
    "error": "Liquidity check is not applicable to this input."  
  }  
}
```

Status Values

The standard status values should be:

- pass — the check completed successfully.
- fail — the check was attempted but failed.
- skip — the check was not applicable to the submitted input.

These values should be used consistently across all checks.

6. Blockchain Evidence

Where available, the JSON should retain the relevant evidence gathered from blockchain sources.

This may include:

Contract Information

- Contract address.
- Bytecode.
- Verified ABI where available.
- Contract creator.
- Contract creation transaction.

Transaction Information

- Transaction hash.
- From address.
- To address.
- Direction.
- Value.
- Timestamp.
- Transaction status.

Token Information

- Token activity.
- Token balances/information where available.
- Token transfer information.

Liquidity Information

- Pool address.
- DEX.
- Token pair.
- Liquidity value.
- Relevant liquidity activity where available.

The information should remain structured so that it can later be supplied to the scam detector or LLM.

7. Risk Evaluation

When a scam assessment has been performed, the JSON must contain a standardised risk_evaluation section.

Example:

```
"risk_evaluation": {
```

```

"label": "scam",
"risk_type": "rug_pull",
"confidence": 0.82,
"source": "scam_checker",
"evidence": [
  {
    "description": "Large liquidity removal detected shortly after pool creation.",
    "weight": 0.7
  },
  {
    "description": "Contract source code is not verified.",
    "weight": 0.4
  }
],
"explanation": "The address shows several indicators associated with rug-pull activity."
}

```

Required Risk Fields

label

The overall assessment.

Standard values should include:

- scam
- not_scam
- insufficient_evidence

risk_type

The detected scam/risk category where known.

Example:

```
"risk_type": "rug_pull"
```

If a specific risk type cannot be determined, use an agreed value such as:

```
"risk_type": "unknown"
```

confidence

A numerical representation of confidence in the result.

The value should use a consistent scale between:

0.0 and 1.0

For example:

"confidence": 0.82

represents 82% confidence.

source

The output should identify where the assessment originated.

For example:

- external_detector
- scam_checker

This allows the application to distinguish between an external configured detector and the built-in LLM Scam Checker.

evidence

Evidence should be represented as an array so multiple reasons can be supplied.

Each evidence item should contain:

- A human-readable description.
- An evidence weight/importance where available.

explanation

The risk evaluation should contain a plain-language explanation that can later be displayed through the chat interface.

8. Insufficient Evidence

The JSON must support situations where a reliable scam assessment cannot be made.

Example:

```
"risk_evaluation": {  
  "label": "insufficient_evidence",  
  "risk_type": "unknown",  
  "confidence": 0.0,  
  "source": "scam_checker",  
  "evidence": [],  
  "explanation": "There is not enough available blockchain evidence to provide a reliable  
assessment."  
}
```

The system should not convert missing information into either a scam or non-scam result.

9. Error Handling

An error in one API/check should not invalidate the entire JSON response.

Each API/check should report errors independently.

Example:

```
"transactions": {  
  "status": "fail",  
  "source": "Etherscan",  
  "data": null,  
  "error": "Unable to retrieve transaction history."  
}
```

A skipped check should also provide a reason:

```
"rugcheck": {  
  "status": "skip",  
  "source": "RugCheck",  
  "data": null,  
  "error": "RugCheck is not applicable to this blockchain."  
}
```

Error messages should:

- Clearly identify which check failed.
- Provide a readable reason where available.
- Not remove successful results from other checks.
- Never silently hide missing information.

10. Summary

The combined JSON should provide a simple summary of completed API checks.

Example:

```
"summary": {  
  "passed": 4,  
  "failed": 1,  
  "skipped": 2  
}
```

This allows other components to quickly determine whether all required information was successfully retrieved.

11. Example Combined Output

```
{
  "input": {
    "value": "0xABC123...",
    "input_type": "address",
    "chain": "ethereum"
  },

  "address_analysis": {
    "chain_family": "evm",
    "address_type": "token_contract"
  },

  "api_checks": {
    "contract": {
      "status": "pass",
      "source": "Etherscan",
      "data": {},
      "error": null
    },

    "transactions": {
      "status": "pass",
      "source": "Etherscan",
      "data": [],
      "error": null
    },

    "honeypot": {
      "status": "pass",
      "source": "Honeypot",
      "data": {},
      "error": null
    },

    "liquidity": {
      "status": "pass",
      "source": "DexScreener",
      "data": [],
      "error": null
    }
  },

  "risk_evaluation": {
    "label": "scam",
    "risk_type": "rug_pull",
    "confidence": 0.82,
```

```

"source": "scam_checker",
"evidence": [
  {
    "description": "Large liquidity removal detected shortly after pool creation.",
    "weight": 0.7
  }
],
"explanation": "The available blockchain evidence indicates a high risk of rug-pull activity."
},

"summary": {
  "passed": 4,
  "failed": 0,
  "skipped": 0
}
}

```

12. JSON Output Acceptance Criteria

- JSON uses a consistent top level structure.
- Original input and blockchain/network are identified.
- Address/input type is included.
- APIs/checks performed are identifiable.
- Each API check includes a pass, fail or skip status.
- Successful checks retain their returned data.
- Failed/skipped checks include an explanatory error/reason.
- One failed API does not remove successful information from other APIs.
- Scam assessments include a standard risk label.
- Scam assessments include risk type where available.
- Scam assessments include a confidence value.
- Scam assessments identify whether the result came from an external detector or the Scam Checker.
- Supporting evidence can contain multiple evidence items.
- A plain-language explanation can be included with the assessment.
- insufficient_evidence can be represented without incorrectly classifying the input as safe or a scam.
- Summary counts for passed, failed and skipped checks are included.